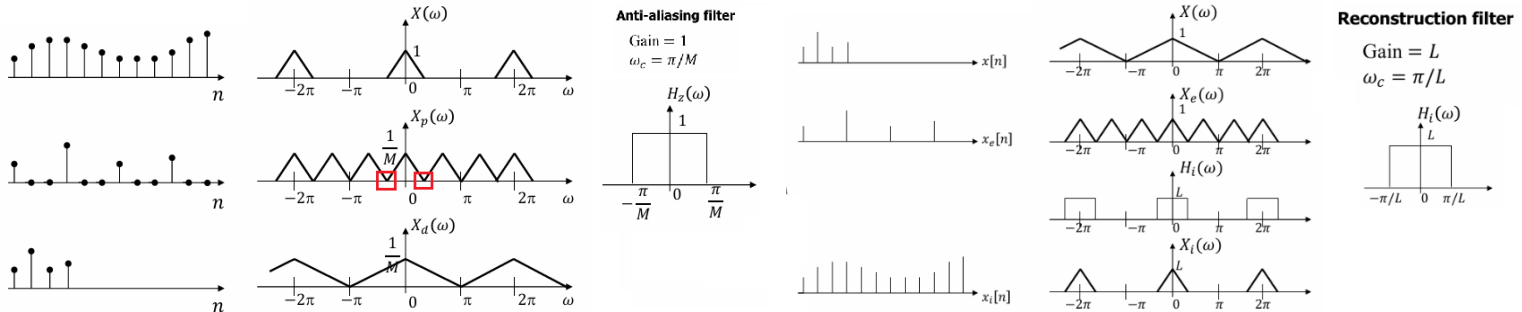


1. Explanation of the entire pipeline

1.1. Brief Explanation of Decimation and Interpolation



Decimation is a technique for reducing a signal's sampling rate.

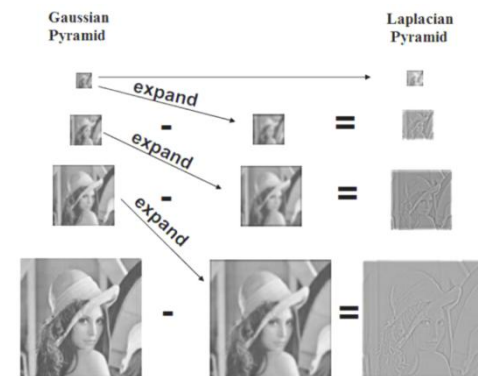
When the sampling rate is lowered, the size of a 2D image decreases accordingly. In this Assignment1, we'll shrink the height and width to half through downsampling. However, in the red boxed section of the Decimation diagram on the left, if the signal's frequencies overlap, aliasing can occur. To prevent aliasing during downsampling, an anti-aliasing filter is essential. For this assignment, we'll design and implement this filter using a 2D Gaussian filter.

Interpolation is a technique for increasing a signal's sampling rate, which, in the case of a 2D image, results in an increased image size. New samples are inserted with a value of 0 to extend the signal's length, and then an appropriate reconstruction filter is applied to estimate and fill in suitable values for these inserted samples. Additionally, when applying the reconstruction filter, the energy of the original signal must be preserved, so a reconstruction filter with a gain equal to the sampling rate is used. In this Assignment1, a 2D Gaussian filter with a gain of 4 is used to upsample the image, doubling both the height and width.

1.2. Brief Explanation of Gaussian Pyramid and Laplacian Pyramid

A Gaussian Pyramid is a method of creating a hierarchical image structure in a pyramid form by repeatedly applying blur and downsampling. Here, the blur serves the same role as the anti-aliasing filter described earlier, which prevents aliasing. Using a Gaussian Pyramid, you can obtain lower-resolution images in a hierarchical manner, but due to the blurring process, high-frequency information is lost.

The Laplacian Pyramid is a method for extracting and storing the high-frequency components of an image by saving the differences between images generated in a Gaussian Pyramid. By examining the Lena image example, you can see how the Laplacian Pyramid is created. This approach allows the high-frequency components lost in the Gaussian Pyramid to be preserved. When reconstructing an image, combining the Gaussian Pyramid and the Laplacian Pyramid enables reconstruction with minimal information loss. Gaussian Pyramid and Laplacian Pyramid can be expressed as follows:



Gaussian Pyramid: $G_{i+1} = \text{downsample}(I_{\text{blurred}}) = \text{downsample}(I * G_i)$

Laplacian Pyramid: $L_i = G_i - \text{upsample}(G_{i+1})$

1.3. Brief Explanation of Assignment 1 Pipeline

In Assignment 1, when there is no noise, we perform the process of building a Gaussian Pyramid followed by building a Laplacian Pyramid to verify correct operation. Then, we execute the sequence of adding Gaussian noise, building a Gaussian Pyramid, building a Laplacian Pyramid, and denoising.

Since building the Gaussian Pyramid and Laplacian Pyramid was explained in section 1.2, image reconstruction and denoising are covered in section 1.3.

Image reconstruction allows us to restore lost high-frequency components from the Gaussian Pyramid process by following its procedure. Denoising is performed by multiplying the Laplacian Pyramid by a gain and reconstructing, reducing the influence of fine high-frequency components (e.g., L0, L1) to minimize the impact of noise contained in detailed high-frequency information while preserving some high-frequency components.

2. Summary of each function

2.1. gaussian_kernel_1d()

This function generates a 1D Gaussian filter. For images, a 2D Gaussian filter is required, but this can be achieved by applying the 1D Gaussian filter first horizontally and then vertically, producing the same effect as a 2D Gaussian filter. By default, when $\sigma = 1$, the 1x3 1D Gaussian filter is defined as $[1, 2, 1]/4$. In Assignment 1, a Gaussian filter with $\sigma = 1$ is applied to the image to perform downsampling and upsampling.

2.2. sep_conv(img, k1d)

This function performs a convolution operation between a 1D Gaussian filter and an image, first in the horizontal direction and then in the vertical direction, producing the same result as a 2D convolution with a 2D Gaussian filter. To maintain the image size, it applies reflection padding of size 1 in the horizontal direction before performing the 1D convolution, and then applies reflection padding of size 1 in the vertical direction before performing another 1D convolution.

The process of performing 1D convolutions in horizontal and vertical directions can be derived as follows.

$$G_{2D} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}, G_h = \frac{1}{4} [1 \quad 2 \quad 1], G_v = \frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

$$G_v * G_h = G_v * G_h * \text{Impulse} = \left(\frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \right) * \left(\frac{1}{4} [1 \quad 2 \quad 1] \right) * \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \frac{1}{16} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 \\ 1 & 2 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \frac{1}{16} \begin{bmatrix} 1 \cdot 1 & 1 \cdot 2 & 1 \cdot 1 \\ 2 \cdot 1 & 2 \cdot 2 & 2 \cdot 1 \\ 1 \cdot 1 & 1 \cdot 2 & 1 \cdot 1 \end{bmatrix} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$G_v * (G_h * \text{Image}) = (G_v * G_h) * \text{Image} = G_{2D} * \text{Image} \quad (\text{since, convolution satisfies the associative law, LTI system})$$

2.3. downsample2(img, k1d)

As described in section 1.1, this function applies a Gaussian filter to blur the image and scales the height and width by half. The output is generated by selecting only the even-indexed rows and columns.

2.4. upsample2(img, k1d)

As described in section 1.1, this function doubles the height and width of the image and inserts zeros into the new samples. Since the height and width are doubled, a Gaussian filter with a gain of 4 is used to perform reconstruction, replacing the inserted zeros with appropriate values.

2.5. build_gaussian_pyramid(img, levels)

This function converts the input image, given in uint8 format, to float values between 0 and 1. It then iteratively performs downsample2 for the number of levels specified by the parameter and appends the results to a list, thereby generating a Gaussian Pyramid that holds 2D image data as float values. Gaussian filter is generated by gaussian_kernel_1d() function.

2.6. build_laplacian_pyr(G)

For each level of the Gaussian Pyramid, it computes $L_i = G_i - \text{upsample}(G_{i+1})$ to extract high-frequency components and construct the Laplacian Pyramid. To ensure the shapes of G_i and $\text{upsample}(G_{i+1})$ match, $\text{upsample}(G_{i+1})$ is sliced to have the same shape as G_i .

2.7. reconstruct_from_lap(Ls, G_top)

This function reconstructs the image at each level through the process $G_i = L_i + \text{Upsample}(G_{i+1})$. The Gaussian filter is generated using the previously described gaussian_kernel_1d() function. To ensure the shapes of L_i and $\text{upsample}(G_{i+1})$ are aligned, $\text{upsample}(G_{i+1})$ is sliced to match the shape of L_i .

2.8. lap_pyr_filter(img, gains, levels=4)

Using the provided image and levels, this function generates a Gaussian Pyramid and a Laplacian Pyramid. To reduce noise, it applies the lowest gain values to the Laplacian Pyramid elements representing detailed high-frequency information (e.g., L0, L1) and higher gain values to subsequent levels, thereby preserving some high-frequency information while performing noise reduction.

2.9. add_gaussian_noise(img, sigma=0.05, seed=0)

This function generates Gaussian noise using the provided sigma value and adds it to the image, outputting the noisy image and the sigma value. Since the image data must remain between 0 and 1, the noisy image is clipped to stay within this range after adding the noise.

2.10. `add_salt_pepper(img, p=0.02, seed=0)`

This function generates random numbers from a uniform distribution with the same size as the image. If a number is less than p , the corresponding image pixel is set to black; if it is greater than p , the pixel is set to white, producing an image with salt-and-pepper noise.

2.11. `psnr(x, y)`

This function calculates the Peak Signal-to-Noise Ratio (PSNR) between two images x and y , using the following formula:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2, \quad \text{PSNR} = \begin{cases} 99.0 & \text{if MSE} = 0 \\ 10 \cdot \log_{10} \left(\frac{1}{\text{MSE}} \right) & \text{otherwise} \end{cases}$$

2.12. `load_gray(path)`

This function converts an RGB image, with its 3 channels, into a grayscale image by multiplying the pixel values of the red, green, and blue channels by 0.299, 0.587, and 0.114, respectively, and summing them. The original image is first converted to float values between 0 and 1, and the output is also clipped to remain within the 0 to 1 range.

2.13. `to_float01(img)`

This function converts an existing uint8 image to float values between 0 and 1. The float type allows for more precise numerical representation compared to uint8 and enables normalization between 0 and 1. As a result, it is more accurate in tasks such as adding noise or calculating PSNR during image processing.

2.14. Why are these functions necessary?

Functions closely related to Gaussian, Laplacian Pyramid such as `upsample2`, `downsample2` are needed to generate pyramid. Utility functions such as `psnr`, `add_gaussian_noise` are needed to simulate and evaluate image filtering. Also, `load_gray` is needed to process simple image filtering which only contains 1channel.

3. Image explanations

Due to the page limit, only the PSNR results are included in the report. Images are included in the zip file. (seed = 42)

	Noisy (Setting 1)	Denoised (Setting 1)	Noisy (Setting 2)	Denoised (Setting 2)
sigma = 0.01	40.00 dB	28.52 dB	40.00 dB	33.02 dB
sigma = 0.05	26.06 dB	27.52 dB	26.06 dB	28.40 dB
sigma = 0.1	20.21 dB	25.43 dB	20.21 dB	23.70 dB
p = 0.01	25.09 dB	27.24 dB	25.09 dB	27.67 dB
p = 0.02	22.06 dB	26.19 dB	22.06 dB	25.26 dB
p = 0.05	18.09 dB	24.00 dB	18.09 dB	21.68 dB

Setting 1 uses levels = 5 and gains = [0.35, 0.50, 0.75, 0.95, 1.00] for processing, while Setting 2 uses levels = 4 and gains = [0.60, 0.75, 0.90, 1.00]. The differences between Setting 1 and Setting 2 lie in how finely the levels are divided and how low the starting gain values are for the Laplacian Pyramid elements representing detailed high-frequency information (e.g., L0, L1).

From these results, it can be observed that for stronger noise, applying stronger blurring to the elements representing detailed high-frequency information (e.g., L0, L1) significantly reduces the noise's impact, making it effective for noise removal. Additionally, Setting 1, with its more finely divided levels compared to Setting 2, allows for more precise adjustments during high-frequency filtering.

On the other hand, when noise is not strong, applying higher gain values to the elements representing detailed high-frequency information (e.g., L0, L1) compared to Setting 1 preserves more high-frequency information, resulting in better output images.

When applying the same Gaussian and Laplacian pyramid filtering for denoising both Gaussian noise and salt-and-pepper noise, it can be observed that the PSNR values tend to increase, indicating effective denoising for both types of noise.

However, when examining the actual images, salt-and-pepper noise remains visible in the denoised image, though its intensity is reduced, showing that the denoising is less effective compared to Gaussian noise. Therefore, salt-and-pepper noise requires the use of a more effective denoising filter, such as a median filter.

4. References: <https://m.blog.naver.com/koreadeep> Sampling_Rate_Change.pdf(by prof.강문기) lecture2 Image processing.pdf (by prof.김종유)